

Within–Between Decomposition for Repeated-Measure Algorithm Validation

Junrui

2026-06-16

Contents

Goal	2
Mathematical setup	2
Why standardization is not required for testing $\beta_B = \beta_W$	3
Pooled model and equality of slopes	3
Reparameterized model	4
Simulation setup	4
Load packages	4
Simulation function	5
Model fitting function	6
Summary function	6
Simulate three scenarios	7
Fit models	8
Summarize main results	8
Interpretation of model coefficients	9
Inspect model summaries	9
Contrast test using <code>glht()</code>	14
Clean table from <code>glht()</code> results	17
Compare <code>glht()</code> with reparameterized model	17
Clean pooled-model comparison table	18
Plot raw repeated-measure relationship	19
Plot within-subject relationship	21

Plot between-subject relationship	24
Optional: compare unstandardized and standardized effects	26
Scenario-specific interpretation	27
Scenario 1: Opposite direction	27
Scenario 2: Same direction, different size	27
Scenario 3: Same direction, similar size	27
Practical recommendation	28

Goal

Suppose we have repeated measurements for two variables:

- Y_{ij} : ground truth measurement for subject i at time j
- X_{ij} : algorithm prediction of the ground truth for subject i at time j

The scientific question is whether the algorithm prediction X tracks the ground truth Y over time.

A key complication is that both X and Y may vary across subjects and within subjects over time. A single pooled association between X and Y can therefore mix two different sources of information:

1. **Between-subject association:** Do subjects with higher average algorithm predictions also have higher average ground truth values?
2. **Within-subject association:** When a subject's algorithm prediction is higher than their own usual level, is their ground truth also higher?

This distinction is important for avoiding Simpson's paradox, ecological fallacy, or symptom-paradox-type interpretation problems.

Mathematical setup

For each subject i , define the subject-level mean of X as:

$$\bar{X}_i = \frac{1}{n_i} \sum_{j=1}^{n_i} X_{ij}$$

and the within-subject deviation as:

$$X_{ij} - \bar{X}_i$$

Then X_{ij} can be decomposed as:

$$X_{ij} = \bar{X}_i + (X_{ij} - \bar{X}_i)$$

A within-between mixed model can be written as:

$$Y_{ij} = \beta_0 + \beta_B \bar{X}_i + \beta_W (X_{ij} - \bar{X}_i) + \gamma t_{ij} + b_i + \epsilon_{ij}$$

where:

- β_B is the **between-subject slope**
- β_W is the **within-subject slope**

- t_{ij} is time
- b_i is a subject-level random intercept
- ϵ_{ij} is residual error

The between-subject slope answers:

Do people with higher average algorithm predictions also have higher average ground truth values?

The within-subject slope answers:

When the same person's algorithm prediction is above their own average, is their ground truth also above their own average?

For algorithm validation, β_W is often particularly important because it reflects within-person tracking over time.

Why standardization is not required for testing $\beta_B = \beta_W$

Although $\bar{X}_i$ and $X_{ij} - \bar{X}_i$ often have different variances, they have the same measurement unit.

For example, suppose X is an algorithm-predicted symptom score measured in points. Then:

- X_{ij} is measured in symptom-score points
- $\bar{X}_i$ is measured in symptom-score points
- $X_{ij} - \bar{X}_i$ is also measured in symptom-score points

Therefore:

$$\beta_B = \frac{\text{units of } Y}{\text{one point of } X}$$

and:

$$\beta_W = \frac{\text{units of } Y}{\text{one point of } X}$$

So the contrast:

$$H_0 : \beta_B - \beta_W = 0$$

is meaningful on the original scale.

Standardizing $\bar{X}_i$ and $X_{ij} - \bar{X}_i$ separately would answer a different question:

Is a one-standard-deviation difference in between-subject X associated with the same change in Y as a one-standard-deviation within-subject deviation in X ?

That can be useful descriptively, but it is not the same as testing whether a single raw-unit pooled slope is justified.

Pooled model and equality of slopes

The pooled model is:

$$Y_{ij} = \beta_0 + \beta X_{ij} + \gamma t_{ij} + b_i + \epsilon_{ij}$$

This model implicitly assumes that the between-subject and within-subject slopes are the same:

$$\beta_B = \beta_W = \beta$$

So before using a pooled model, it is useful to test:

$$H_0 : \beta_B = \beta_W$$

or equivalently:

$$H_0 : \beta_B - \beta_W = 0$$

Reparameterized model

An algebraically equivalent model is:

$$Y_{ij} = \beta_0 + \beta_W X_{ij} + \delta \bar{X}_i + \gamma t_{ij} + b_i + \epsilon_{ij}$$

where:

$$\delta = \beta_B - \beta_W$$

Therefore, testing whether the coefficient of $\bar{X}_i$ is zero in this reparameterized model is equivalent to testing:

$$H_0 : \beta_B = \beta_W$$

This is usually the cleanest model-based test.

Simulation setup

We simulate three scenarios:

1. **Opposite direction:** $\beta_B > 0$ and $\beta_W < 0$
This is the textbook Simpson's paradox / symptom paradox case.
2. **Same direction, different size:** $\beta_B > 0$ and $\beta_W > 0$, but $\beta_B \neq \beta_W$
This means the algorithm is directionally consistent, but between-subject and within-subject associations differ materially.
3. **Same direction, similar size:** $\beta_B \approx \beta_W > 0$
This is the case where a pooled model may be more defensible.

Load packages

```
library(lme4)
library(lmerTest)
library(multcomp)
library(ggplot2)
library(dplyr)
library(broom.mixed)
```

Simulation function

```
simulate_within_between <- function(
  n_id = 120,
  n_time = 6,
  beta_B = 1.0,
  beta_W = 1.0,
  beta_time = 0.2,
  sigma_subject = 1.0,
  sigma_error = 1.0,
  mean_x_sd = 2.0,
  within_x_sd = 1.0,
  scenario_name = "scenario"
) {

  dat <- expand.grid(
    id = 1:n_id,
    time = 0:(n_time - 1)
  )

  dat <- dat %>%
    arrange(id, time)

  # Subject-level average X: between-subject component
  x_bar_true <- rnorm(n_id, mean = 0, sd = mean_x_sd)

  # Subject-level random intercept for Y
  subject_re <- rnorm(n_id, mean = 0, sd = sigma_subject)

  dat$x_bar_true <- x_bar_true[dat$id]
  dat$subject_re <- subject_re[dat$id]

  # Within-subject deviation of X
  dat$x_within_true <- rnorm(nrow(dat), mean = 0, sd = within_x_sd)

  # Observed X
  dat$x <- dat$x_bar_true + dat$x_within_true

  # Generate Y using separate between and within slopes
  dat$y <-
    10 +
    beta_B * dat$x_bar_true +
    beta_W * dat$x_within_true +
    beta_time * dat$time +
    dat$subject_re +
    rnorm(nrow(dat), mean = 0, sd = sigma_error)

  # In real data, we usually do not know the true x_bar or true within component.
  # Therefore, we estimate them empirically from observed X.
  dat <- dat %>%
    group_by(id) %>%
    mutate(
      x_bar = mean(x),
```

```

    x_within = x - x_bar
  ) %>%
  ungroup()

dat$scenario <- scenario_name

return(dat)
}

```

Model fitting function

```

fit_models <- function(dat) {

  # Model A: Explicit within-between decomposition
  #  $y = \beta_B * x_{bar} + \beta_W * x_{within} + time + random\ intercept$ 
  m_wb <- lmer(
    y ~ x_bar + x_within + time + (1 | id),
    data = dat
  )

  # Model B: Reparameterized version
  #  $y = \beta_W * x + \delta * x_{bar} + time + random\ intercept$ 
  # where  $\delta = \beta_B - \beta_W$ 
  m_reparam <- lmer(
    y ~ x + x_bar + time + (1 | id),
    data = dat
  )

  # Model C: Pooled model
  # Assumes  $\beta_B = \beta_W$ 
  m_pooled <- lmer(
    y ~ x + time + (1 | id),
    data = dat
  )

  list(
    within_between = m_wb,
    reparameterized = m_reparam,
    pooled = m_pooled
  )
}

```

Summary function

```

summarize_results <- function(models, scenario_label) {

  m_wb <- models$within_between
  m_reparam <- models$reparameterized
  m_pooled <- models$pooled

  wb_coef <- summary(m_wb)$coefficients

```

```

rep_coef <- summary(m_reparam)$coefficients
pooled_coef <- summary(m_pooled)$coefficients

beta_B_hat <- wb_coef["x_bar", "Estimate"]
beta_W_hat <- wb_coef["x_within", "Estimate"]

beta_B_p <- wb_coef["x_bar", "Pr(>|t|)"]
beta_W_p <- wb_coef["x_within", "Pr(>|t|)"]

# In the reparameterized model:
# coefficient of x is beta_W
# coefficient of x_bar is beta_B - beta_W
beta_W_reparam <- rep_coef["x", "Estimate"]
delta_hat <- rep_coef["x_bar", "Estimate"]
delta_p <- rep_coef["x_bar", "Pr(>|t|)"]

pooled_beta <- pooled_coef["x", "Estimate"]
pooled_p <- pooled_coef["x", "Pr(>|t|)"]

out <- data.frame(
  scenario = scenario_label,

  beta_B_hat_decomposed = beta_B_hat,
  beta_W_hat_decomposed = beta_W_hat,
  beta_B_minus_beta_W_from_decomposed = beta_B_hat - beta_W_hat,

  beta_W_hat_reparameterized = beta_W_reparam,
  delta_beta_B_minus_beta_W = delta_hat,
  p_value_test_beta_B_equals_beta_W = delta_p,

  pooled_beta_hat = pooled_beta,
  pooled_beta_p_value = pooled_p
)

return(out)
}

```

Simulate three scenarios

```

set.seed(2026)

# Scenario 1:
# Simpson's paradox / symptom paradox
# Between effect is positive, within effect is negative.
dat1 <- simulate_within_between(
  beta_B = 1.2,
  beta_W = -0.8,
  beta_time = 0.2,
  scenario_name = "1_opposite_direction"
)

# Scenario 2:

```

```

# Same direction, but different magnitude.
dat2 <- simulate_within_between(
  beta_B = 1.2,
  beta_W = 0.3,
  beta_time = 0.2,
  scenario_name = "2_same_direction_different_size"
)

# Scenario 3:
# Same direction, similar magnitude.
dat3 <- simulate_within_between(
  beta_B = 0.8,
  beta_W = 0.75,
  beta_time = 0.2,
  scenario_name = "3_same_direction_similar_size"
)

```

Fit models

```

models1 <- fit_models(dat1)
models2 <- fit_models(dat2)
models3 <- fit_models(dat3)

```

Summarize main results

```

results <- bind_rows(
  summarize_results(models1, "1_opposite_direction"),
  summarize_results(models2, "2_same_direction_different_size"),
  summarize_results(models3, "3_same_direction_similar_size")
)

print(results)

```

```

##              scenario beta_B_hat_decomposed beta_W_hat_decomposed
## 1      1_opposite_direction          1.1654363          -0.7819864
## 2 2_same_direction_different_size          1.1969749           0.2317959
## 3   3_same_direction_similar_size          0.7684597           0.7750300
##  beta_B_minus_beta_W_from_decomposed beta_W_hat_reparameterized
## 1              1.947422763          -0.7819864
## 2              0.965178994           0.2317959
## 3             -0.006570348           0.7750300
##  delta_beta_B_minus_beta_W p_value_test_beta_B_equals_beta_W pooled_beta_hat
## 1              1.947422763          2.178840e-71          -0.6882542
## 2              0.965178994          3.943764e-43           0.4411187
## 3             -0.006570348          9.169088e-01           0.7720828
##  pooled_beta_p_value
## 1          1.325640e-55
## 2          3.106102e-28
## 3          5.164328e-86

```


Interpretation of model coefficients

For the decomposed model:

```
y ~ x_bar + x_within + time + (1 | id)
```

- `x_bar` estimates β_B
- `x_within` estimates β_W

For the reparameterized model:

```
y ~ x + x_bar + time + (1 | id)
```

- `x` estimates β_W
- `x_bar` estimates $\beta_B - \beta_W$

For the pooled model:

```
y ~ x + time + (1 | id)
```

- `x` estimates a single pooled slope, implicitly assuming $\beta_B = \beta_W$

Inspect model summaries

```
summary(models1$within_between)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ x_bar + x_within + time + (1 | id)
##      Data: dat
##
## REML criterion at convergence: 2339
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -3.15686 -0.63456 -0.01799  0.63214  2.43490
##
## Random effects:
##  Groups   Name                Variance Std.Dev.
##  id       (Intercept)         1.7468   1.3217
##  Residual                    0.9844   0.9922
## Number of obs: 720, groups: id, 120
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)  10.17231    0.13783 163.94256  73.801 < 2e-16 ***
## x_bar         1.16544    0.06209 118.00000  18.769 < 2e-16 ***
## x_within     -0.78199    0.04049 598.00000 -19.313 < 2e-16 ***
## time          0.17518    0.02166 598.00000   8.087 3.42e-15 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##              (Intr) x_bar  x_wthn
## x_bar         0.086
## x_within     -0.013  0.000
```

```
## time      -0.393  0.000  0.033
summary(models1$reparameterized)

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ x + x_bar + time + (1 | id)
## Data: dat
##
## REML criterion at convergence: 2339
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -3.15686 -0.63456 -0.01799  0.63214  2.43490
##
## Random effects:
## Groups Name Variance Std.Dev.
## id      (Intercept) 1.7468  1.3217
## Residual              0.9844  0.9922
## Number of obs: 720, groups: id, 120
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept) 10.17231    0.13783 163.94256  73.801 < 2e-16 ***
## x           -0.78199    0.04049 598.00000 -19.313 < 2e-16 ***
## x_bar        1.94742    0.07413 231.43525  26.271 < 2e-16 ***
## time         0.17518    0.02166 598.00000   8.087 3.42e-15 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) x      x_bar
## x      -0.013
## x_bar  0.079 -0.546
## time  -0.393  0.033 -0.018
summary(models1$pooled)

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ x + time + (1 | id)
## Data: dat
##
## REML criterion at convergence: 2596.1
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -3.13937 -0.60977 -0.06205  0.63750  2.55402
##
## Random effects:
## Groups Name Variance Std.Dev.
## id      (Intercept) 16.0467  4.0058
## Residual              0.9932  0.9966
## Number of obs: 720, groups: id, 120
##
```

```

## Fixed effects:
##           Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)   9.81329    0.37161 113.65163  26.408 < 2e-16 ***
## x             -0.68825    0.03968 645.04583 -17.345 < 2e-16 ***
## time          0.17684    0.02176 587.11613   8.128 2.6e-15 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) x
## x      0.016
## time -0.146  0.032

summary(models2$within_between)

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ x_bar + x_within + time + (1 | id)
## Data: dat
##
## REML criterion at convergence: 2289.7
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.77215 -0.67032 -0.00928  0.55023  2.88163
##
## Random effects:
## Groups Name Variance Std.Dev.
## id      (Intercept) 0.8091  0.8995
## Residual          1.0340  1.0169
## Number of obs: 720, groups: id, 120
##
## Fixed effects:
##           Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)   9.72684    0.10610 217.43416  91.675 < 2e-16 ***
## x_bar         1.19697    0.04430 117.99994  27.022 < 2e-16 ***
## x_within      0.23180    0.04242 598.00003   5.465 6.81e-08 ***
## time          0.19445    0.02219 598.00003   8.762 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) x_bar x_wthn
## x_bar    -0.009
## x_within  0.006  0.000
## time     -0.523  0.000 -0.012

summary(models2$reparameterized)

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ x + x_bar + time + (1 | id)
## Data: dat
##
## REML criterion at convergence: 2289.7

```

```

##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.77215 -0.67032 -0.00928  0.55023  2.88163
##
## Random effects:
##   Groups   Name                Variance Std.Dev.
##   id       (Intercept) 0.8091    0.8995
##   Residual                1.0340    1.0169
## Number of obs: 720, groups: id, 120
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)   9.72684    0.10610 217.43416  91.675 < 2e-16 ***
## x              0.23180    0.04242 598.00003   5.465 6.81e-08 ***
## x_bar          0.96518    0.06133 371.90691  15.737 < 2e-16 ***
## time           0.19445    0.02219 598.00003   8.762 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) x      x_bar
## x      0.006
## x_bar -0.011 -0.692
## time  -0.523 -0.012  0.008
summary(models2$pooled)

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ x + time + (1 | id)
##   Data: dat
##
## REML criterion at convergence: 2457.4
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.91818 -0.63573 -0.01335  0.57728  2.60831
##
## Random effects:
##   Groups   Name                Variance Std.Dev.
##   id       (Intercept) 3.201    1.789
##   Residual                1.076    1.037
## Number of obs: 720, groups: id, 120
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)   9.74690    0.17713 100.40824  55.028 <2e-16 ***
## x              0.44112    0.03829 696.09703  11.521 <2e-16 ***
## time           0.19318    0.02263 545.57495   8.535 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) x

```

```
## x      -0.002
## time -0.319 -0.010

summary(models3$within_between)

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ x_bar + x_within + time + (1 | id)
## Data: dat
##
## REML criterion at convergence: 2312
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.8849 -0.6155 -0.0052  0.6007  3.3693
##
## Random effects:
## Groups Name Variance Std.Dev.
## id      (Intercept) 1.096  1.047
## Residual 1.020  1.010
## Number of obs: 720, groups: id, 120
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept) 10.00153    0.11706 191.80489  85.44 <2e-16 ***
## x_bar        0.76846    0.04677 117.99999  16.43 <2e-16 ***
## x_within     0.77503    0.04210 598.00001  18.41 <2e-16 ***
## time         0.19915    0.02206 598.00000   9.03 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##              (Intr) x_bar x_wthn
## x_bar      -0.090
## x_within -0.016  0.000
## time      -0.471  0.000  0.033
```

```
summary(models3$reparameterized)

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ x + x_bar + time + (1 | id)
## Data: dat
##
## REML criterion at convergence: 2312
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.8849 -0.6155 -0.0052  0.6007  3.3693
##
## Random effects:
## Groups Name Variance Std.Dev.
## id      (Intercept) 1.096  1.047
## Residual 1.020  1.010
## Number of obs: 720, groups: id, 120
```

```
##
## Fixed effects:
##           Estimate Std. Error      df t value Pr(>|t|)
## (Intercept) 10.00153    0.11706 191.80489  85.439  <2e-16 ***
## x           0.77503    0.04210 598.00000  18.408  <2e-16 ***
## x_bar       -0.00657    0.06293 342.35674  -0.104    0.917
## time        0.19915    0.02206 598.00000   9.030  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) x      x_bar
## x      -0.016
## x_bar -0.056 -0.669
## time -0.471  0.033 -0.022
summary(models3$pooled)

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ x + time + (1 | id)
## Data: dat
##
## REML criterion at convergence: 2308.3
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.8837 -0.6155 -0.0064  0.6000  3.3706
##
## Random effects:
## Groups Name Variance Std.Dev.
## id      (Intercept) 1.091    1.044
## Residual          1.020    1.010
## Number of obs: 720, groups: id, 120
##
## Fixed effects:
##           Estimate Std. Error      df t value Pr(>|t|)
## (Intercept) 10.00085    0.11666 194.87758  85.729  <2e-16 ***
## x           0.77208    0.03125 453.84587  24.704  <2e-16 ***
## time        0.19910    0.02204 599.01697   9.033  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) x
## x      -0.072
## time -0.474  0.025
```

Contrast test using `glht()`

The decomposed model estimates β_B and β_W separately. We can use `multcomp::glht()` to test:

$$H_0 : \beta_B - \beta_W = 0$$

In the fitted model, this is:

```
x_bar - x_within = 0
```

```
contrast_beta_B_minus_beta_W_glht <- function(model) {  
  
  # This tests:  
  # H0: beta_B - beta_W = 0  
  #  
  # In the decomposed model:  
  # beta_B = coefficient of x_bar  
  # beta_W = coefficient of x_within  
  
  glht_result <- glht(  
    model,  
    linfct = c("x_bar - x_within = 0")  
  )  
  
  return(glht_result)  
}
```

```
glht1 <- contrast_beta_B_minus_beta_W_glht(models1$within_between)  
glht2 <- contrast_beta_B_minus_beta_W_glht(models2$within_between)  
glht3 <- contrast_beta_B_minus_beta_W_glht(models3$within_between)
```

```
summary(glht1)
```

```
##  
## Simultaneous Tests for General Linear Hypotheses  
##  
## Fit: lmer(formula = y ~ x_bar + x_within + time + (1 | id), data = dat)  
##  
## Linear Hypotheses:  
##               Estimate Std. Error z value Pr(>|z|)  
## x_bar - x_within == 0  1.94742    0.07413   26.27  <2e-16 ***  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
## (Adjusted p values reported -- single-step method)
```

```
confint(glht1)
```

```
##  
## Simultaneous Confidence Intervals  
##  
## Fit: lmer(formula = y ~ x_bar + x_within + time + (1 | id), data = dat)  
##  
## Quantile = 1.96  
## 95% family-wise confidence level  
##  
## Linear Hypotheses:  
##               Estimate lwr      upr  
## x_bar - x_within == 0  1.9474  1.8021 2.0927
```

```
summary(glht2)
```

```
##
```

```
## Simultaneous Tests for General Linear Hypotheses
##
## Fit: lmer(formula = y ~ x_bar + x_within + time + (1 | id), data = dat)
##
## Linear Hypotheses:
##           Estimate Std. Error z value Pr(>|z|)
## x_bar - x_within == 0  0.96518    0.06133   15.74   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## (Adjusted p values reported -- single-step method)
```

```
confint(glht2)
```

```
##
## Simultaneous Confidence Intervals
##
## Fit: lmer(formula = y ~ x_bar + x_within + time + (1 | id), data = dat)
##
## Quantile = 1.96
## 95% family-wise confidence level
##
## Linear Hypotheses:
##           Estimate lwr      upr
## x_bar - x_within == 0 0.9652   0.8450 1.0854
```

```
summary(glht3)
```

```
##
## Simultaneous Tests for General Linear Hypotheses
##
## Fit: lmer(formula = y ~ x_bar + x_within + time + (1 | id), data = dat)
##
## Linear Hypotheses:
##           Estimate Std. Error z value Pr(>|z|)
## x_bar - x_within == 0 -0.00657    0.06293  -0.104   0.917
## (Adjusted p values reported -- single-step method)
```

```
confint(glht3)
```

```
##
## Simultaneous Confidence Intervals
##
## Fit: lmer(formula = y ~ x_bar + x_within + time + (1 | id), data = dat)
##
## Quantile = 1.96
## 95% family-wise confidence level
##
## Linear Hypotheses:
##           Estimate lwr      upr
## x_bar - x_within == 0 -0.00657 -0.12991  0.11677
```


Clean table from glht() results

```
extract_glht_contrast <- function(glht_object, scenario_label) {

  test_sum <- summary(glht_object)
  ci_sum <- confint(glht_object)

  out <- data.frame(
    scenario = scenario_label,
    contrast = names(test_sum$test$coefficients),
    estimate_beta_B_minus_beta_W = as.numeric(test_sum$test$coefficients),
    se = as.numeric(test_sum$test$sigma),
    z_value = as.numeric(test_sum$test$tstat),
    p_value = as.numeric(test_sum$test$pvalues),
    lower_ci = as.numeric(ci_sum$confint[, "lwr"]),
    upper_ci = as.numeric(ci_sum$confint[, "upr"])
  )

  return(out)
}
```

```
glht_results <- bind_rows(
  extract_glht_contrast(glht1, "1_opposite_direction"),
  extract_glht_contrast(glht2, "2_same_direction_different_size"),
  extract_glht_contrast(glht3, "3_same_direction_similar_size")
)

print(glht_results)
```

```
##              scenario      contrast estimate_beta_B_minus_beta_W
## 1      1_opposite_direction x_bar - x_within      1.947422763
## 2 2_same_direction_different_size x_bar - x_within      0.965178994
## 3 3_same_direction_similar_size x_bar - x_within     -0.006570348
##          se    z_value  p_value  lower_ci upper_ci
## 1 0.07412870 26.2708326 0.0000000  1.8021332 2.0927124
## 2 0.06133043 15.7373598 0.0000000  0.8449736 1.0853844
## 3 0.06293116 -0.1044053 0.9168477 -0.1299132 0.1167725
```

Compare glht() with reparameterized model

The `glht()` estimate from the decomposed model should match the `x_bar` coefficient from the reparameterized model:

```
reparam_results <- bind_rows(
  tidy(models1$reparameterized, effects = "fixed") %>%
    filter(term == "x_bar") %>%
    mutate(scenario = "1_opposite_direction"),
  tidy(models2$reparameterized, effects = "fixed") %>%
    filter(term == "x_bar") %>%
    mutate(scenario = "2_same_direction_different_size"),
  tidy(models3$reparameterized, effects = "fixed") %>%
    filter(term == "x_bar") %>%
    mutate(scenario = "3_same_direction_similar_size")
) %>%
```

```

select(scenario, term, estimate, std.error, statistic, p.value)

comparison_glht_reparam <- glht_results %>%
  select(
    scenario,
    glht_estimate = estimate_beta_B_minus_beta_W,
    glht_p_value = p_value
  ) %>%
  left_join(
    reparam_results %>%
      select(
        scenario,
        reparam_estimate = estimate,
        reparam_p_value = p.value
      ),
    by = "scenario"
  )

print(comparison_glht_reparam)

##              scenario glht_estimate glht_p_value reparam_estimate
## 1      1_opposite_direction  1.947422763   0.0000000    1.947422763
## 2 2_same_direction_different_size  0.965178994   0.0000000    0.965178994
## 3  3_same_direction_similar_size -0.006570348   0.9168477   -0.006570348
##   reparam_p_value
## 1    2.178840e-71
## 2    3.943764e-43
## 3    9.169088e-01

```

Clean pooled-model comparison table

```

clean_results <- results %>%
  mutate(
    pooling_decision = case_when(
      abs(delta_beta_B_minus_beta_W) < 0.15 ~ "Pooling likely reasonable",
      TRUE ~ "Do not pool; slopes differ materially"
    )
  )
print(clean_results)

##              scenario beta_B_hat_decomposed beta_W_hat_decomposed
## 1      1_opposite_direction          1.1654363          -0.7819864
## 2 2_same_direction_different_size          1.1969749           0.2317959
## 3  3_same_direction_similar_size          0.7684597           0.7750300
##   beta_B_minus_beta_W_from_decomposed beta_W_hat_reparameterized
## 1              1.947422763          -0.7819864
## 2              0.965178994           0.2317959
## 3             -0.006570348           0.7750300
##   delta_beta_B_minus_beta_W p_value_test_beta_B_equals_beta_W pooled_beta_hat
## 1              1.947422763          2.178840e-71          -0.6882542
## 2              0.965178994          3.943764e-43           0.4411187
## 3             -0.006570348          9.169088e-01           0.7720828
##   pooled_beta_p_value              pooling_decision

```

```
## 1      1.325640e-55 Do not pool; slopes differ materially
## 2      3.106102e-28 Do not pool; slopes differ materially
## 3      5.164328e-86      Pooling likely reasonable
```

The threshold of 0.15 above is only an illustrative rule. In a real validation analysis, the threshold for a practically meaningful difference should be defined based on the measurement scale, clinical context, and intended use of the algorithm.

Plot raw repeated-measure relationship

This plot shows the raw relationship between X and Y .

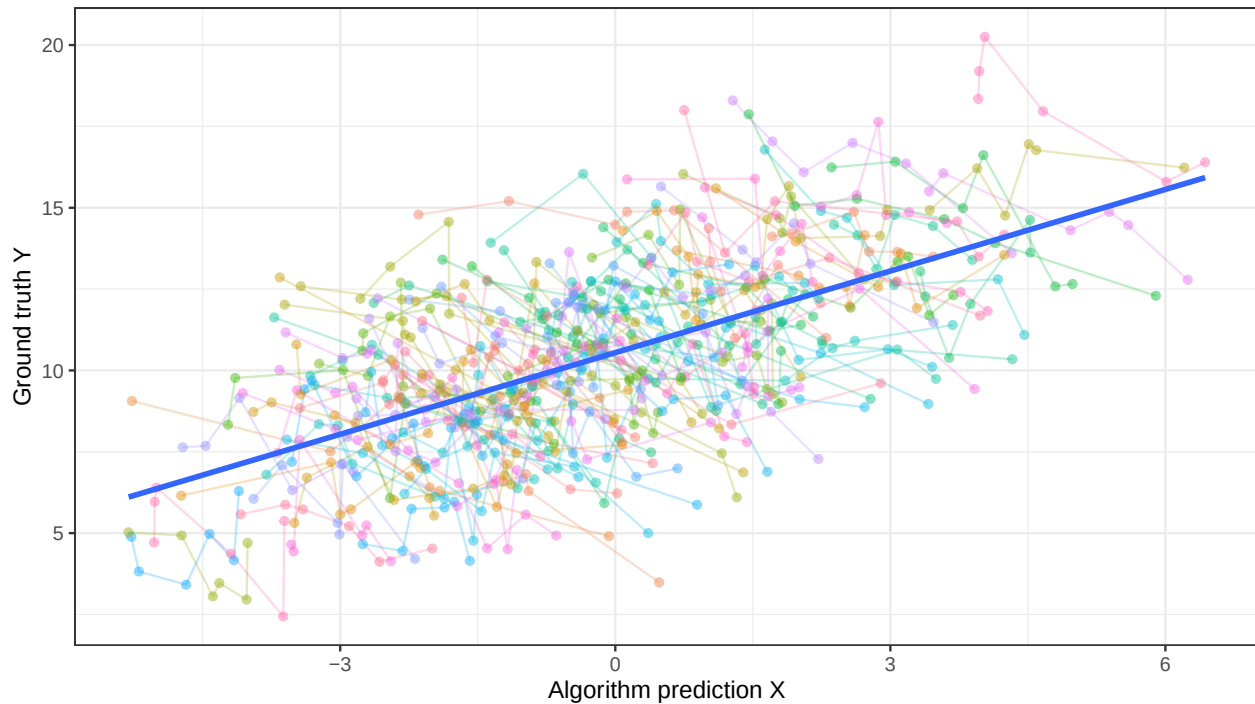
- Thin lines connect repeated measurements within subject.
- The thick smooth line shows the pooled trend.

```
plot_raw_relationship <- function(dat, title_text) {
  dat %>% mutate(id = as.character(id)) %>%
  ggplot(., aes(x = x, y = y, group = id, color = id)) +
    geom_line(alpha = 0.25) +
    geom_point(alpha = 0.45) +
    geom_smooth(
      aes(group = 1),
      method = "lm",
      se = FALSE,
      linewidth = 1.2
    ) +
    guides(color = "none") +
    labs(
      title = title_text,
      subtitle = "Thin lines = within-subject repeated measures; thick line = pooled trend",
      x = "Algorithm prediction X",
      y = "Ground truth Y"
    ) +
    theme_bw()
}
```

```
plot_raw_relationship(dat1, "Scenario 1: Opposite between and within effects")
```

Scenario 1: Opposite between and within effects

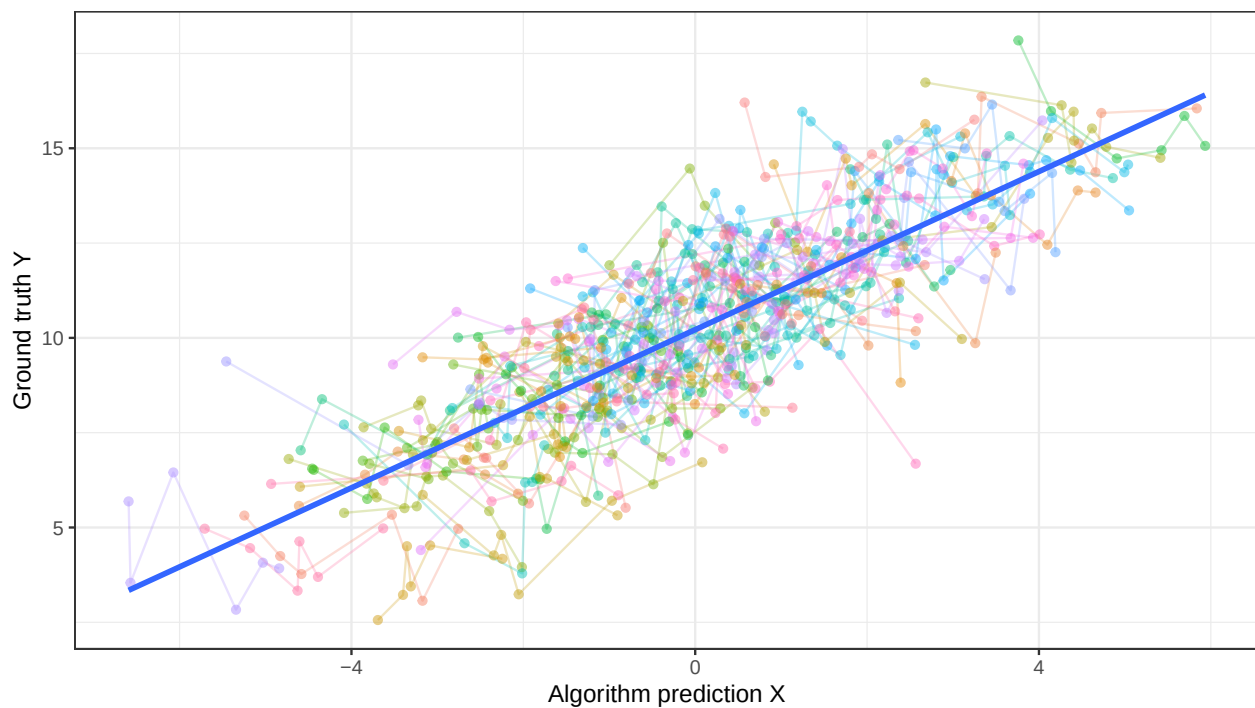
Thin lines = within-subject repeated measures; thick line = pooled trend



```
plot_raw_relationship(dat2, "Scenario 2: Same direction but different magnitude")
```

Scenario 2: Same direction but different magnitude

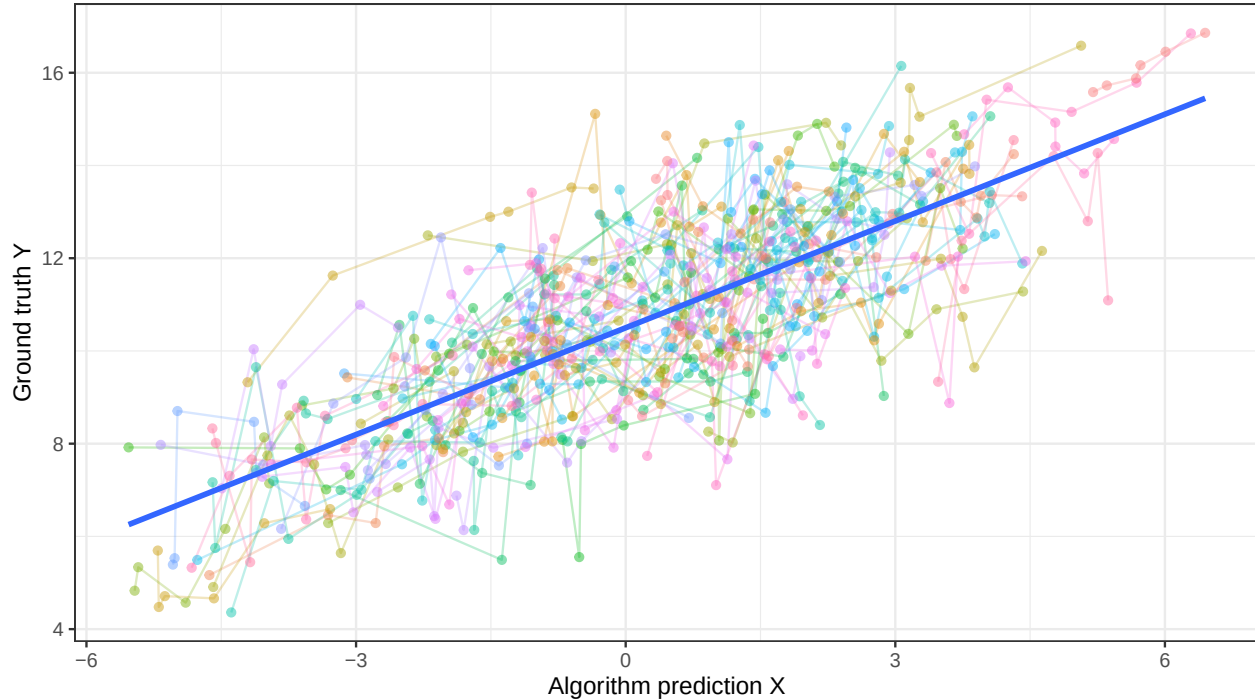
Thin lines = within-subject repeated measures; thick line = pooled trend



```
plot_raw_relationship(dat3, "Scenario 3: Same direction and similar magnitude")
```

Scenario 3: Same direction and similar magnitude

Thin lines = within-subject repeated measures; thick line = pooled trend



Plot within-subject relationship

This plot centers both X and Y within each subject.

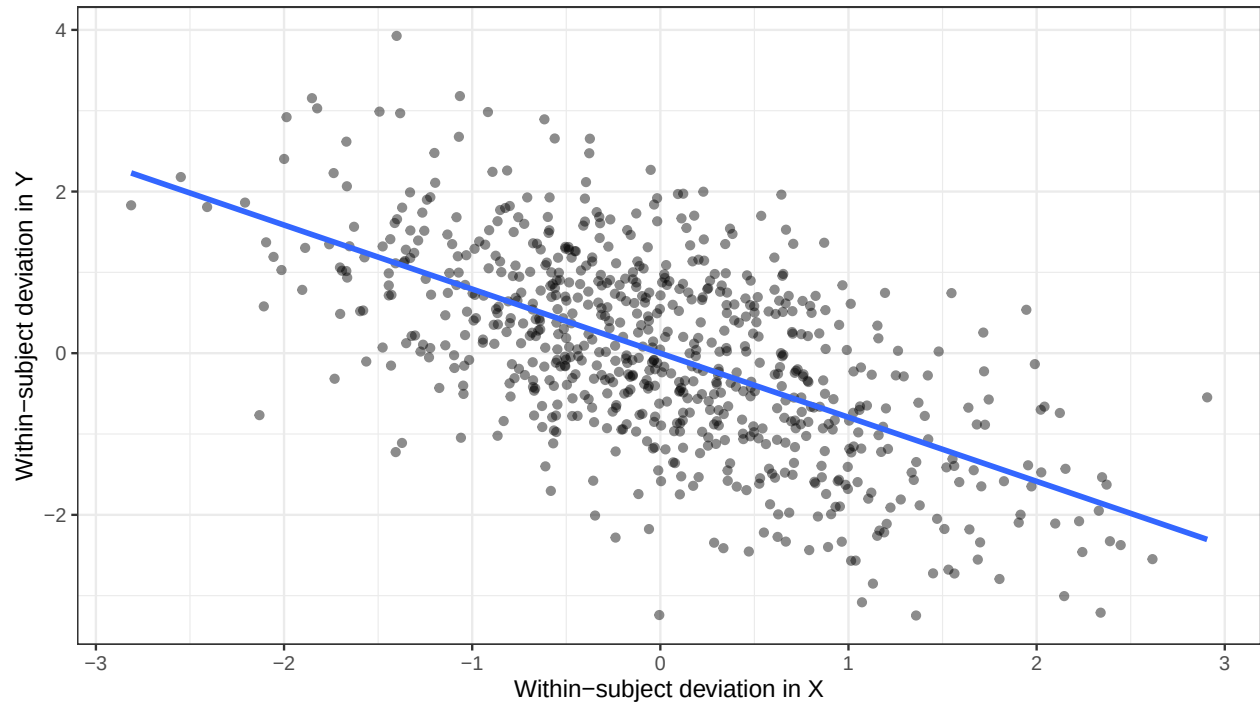
```
plot_within_relationship <- function(dat, title_text) {  
  
  dat <- dat %>%  
    group_by(id) %>%  
    mutate(  
      y_bar = mean(y),  
      y_within = y - y_bar  
    ) %>%  
    ungroup()  
  
  ggplot(dat, aes(x = x_within, y = y_within)) +  
    geom_point(alpha = 0.45) +  
    geom_smooth(  
      method = "lm",  
      se = FALSE,  
      linewidth = 1.2  
    ) +  
    labs(  
      title = title_text,  
      subtitle = "Both X and Y are centered within subject",  
      x = "Within-subject deviation in X",  
      y = "Within-subject deviation in Y"  
    ) +  
    theme_bw()  
}
```

```
}
```

```
plot_within_relationship(dat1, "Scenario 1: Within-subject association")
```

Scenario 1: Within-subject association

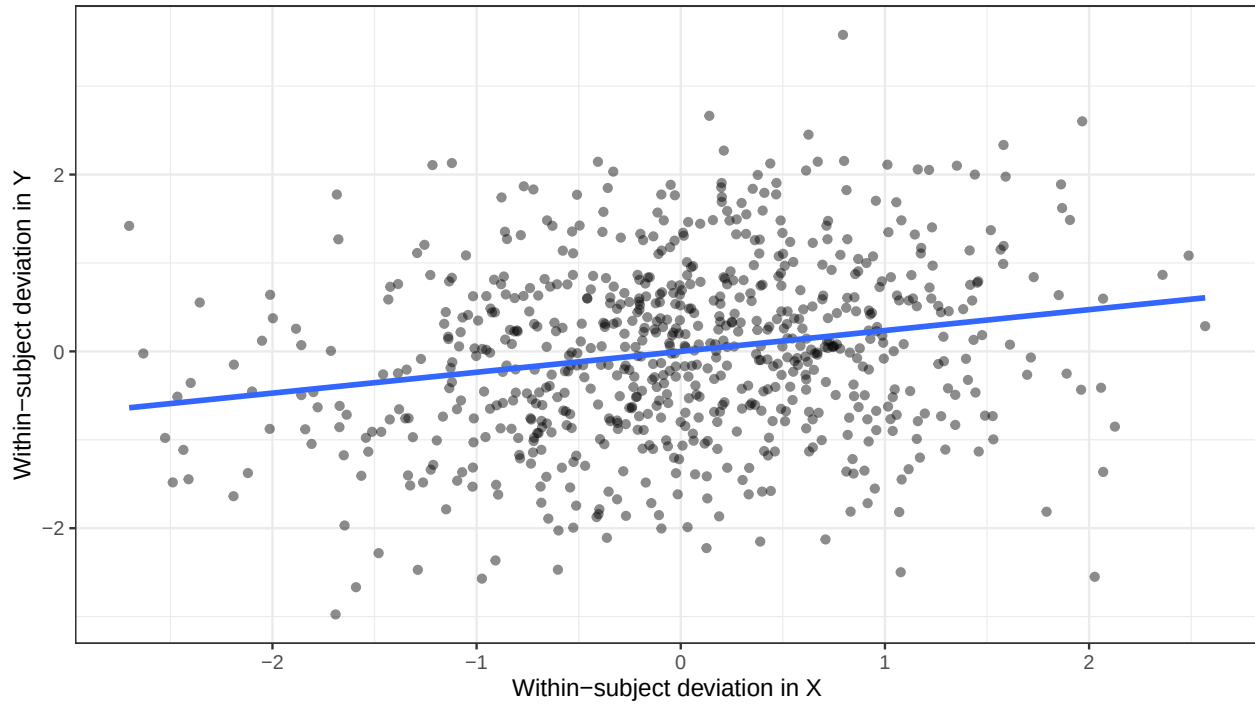
Both X and Y are centered within subject



```
plot_within_relationship(dat2, "Scenario 2: Within-subject association")
```

Scenario 2: Within-subject association

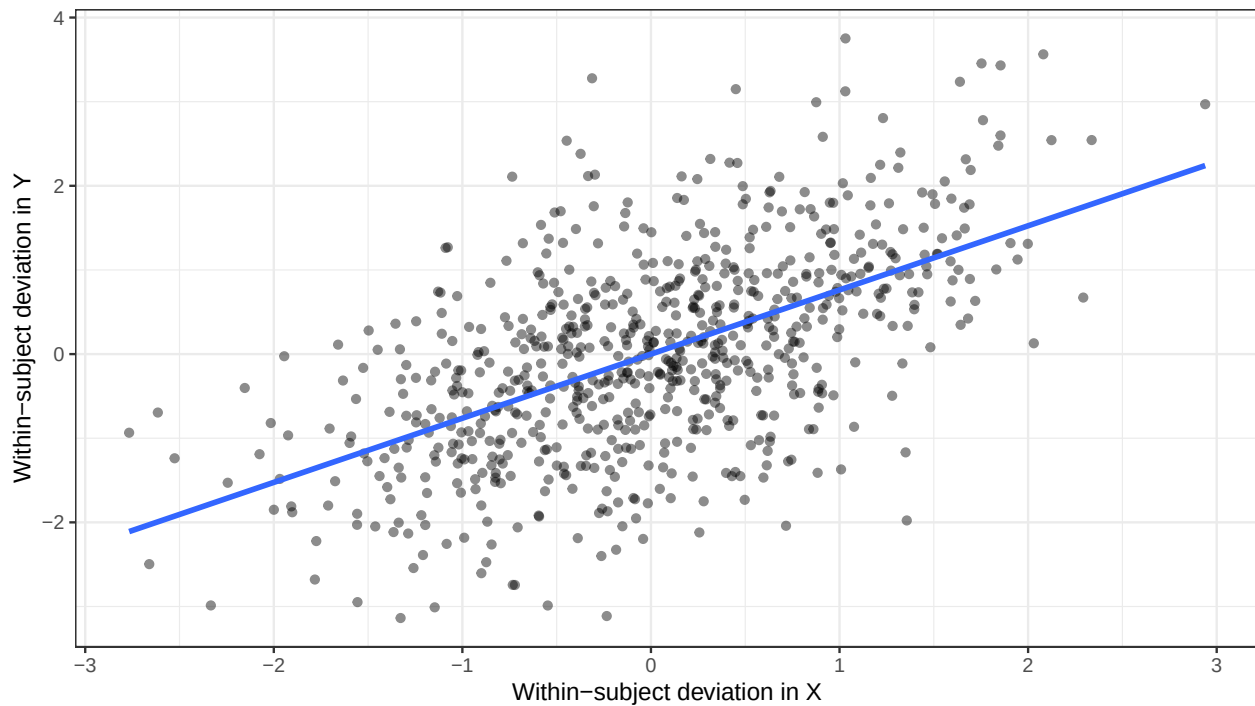
Both X and Y are centered within subject



```
plot_within_relationship(dat3, "Scenario 3: Within-subject association")
```

Scenario 3: Within-subject association

Both X and Y are centered within subject



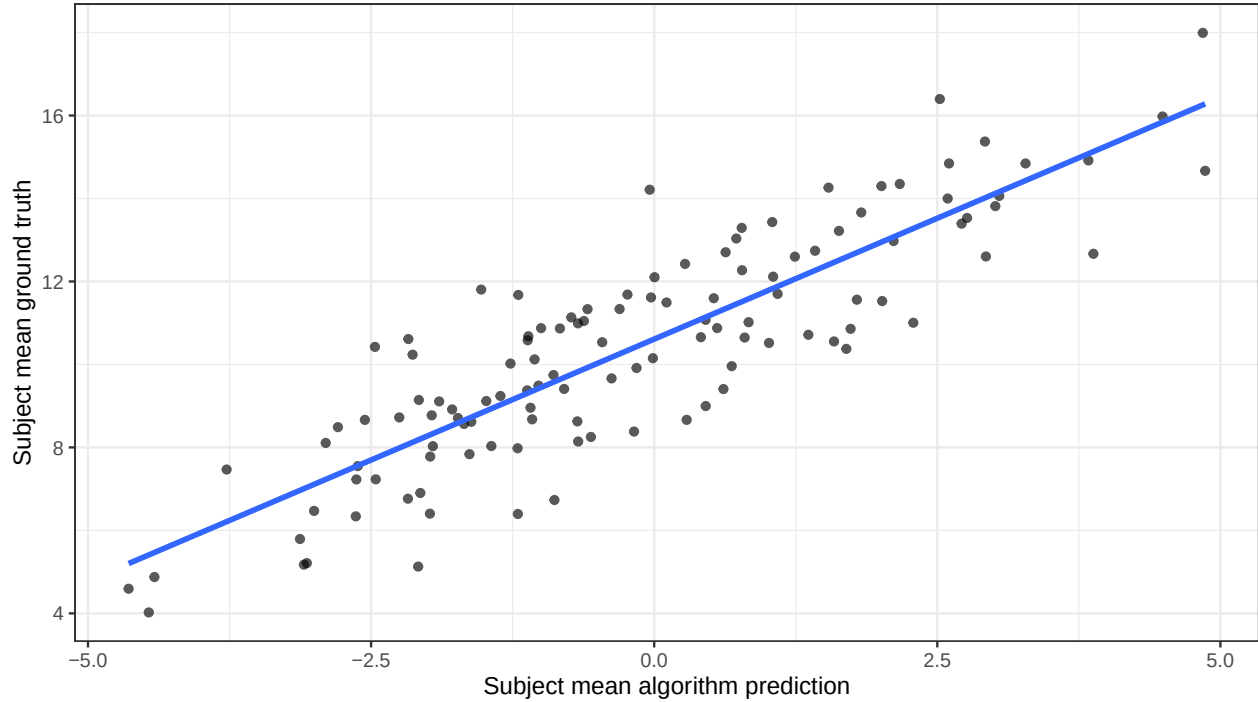
Plot between-subject relationship

This plot collapses each subject to one point.

```
plot_between_relationship <- function(dat, title_text) {  
  
  subject_level <- dat %>%  
    group_by(id) %>%  
    summarize(  
      x_bar = mean(x),  
      y_bar = mean(y),  
      .groups = "drop"  
    )  
  
  ggplot(subject_level, aes(x = x_bar, y = y_bar)) +  
    geom_point(alpha = 0.65) +  
    geom_smooth(  
      method = "lm",  
      se = FALSE,  
      linewidth = 1.2  
    ) +  
    labs(  
      title = title_text,  
      subtitle = "Each point is one subject's average X and average Y",  
      x = "Subject mean algorithm prediction",  
      y = "Subject mean ground truth"  
    ) +  
    theme_bw()  
}  
  
plot_between_relationship(dat1, "Scenario 1: Between-subject association")
```


Scenario 1: Between-subject association

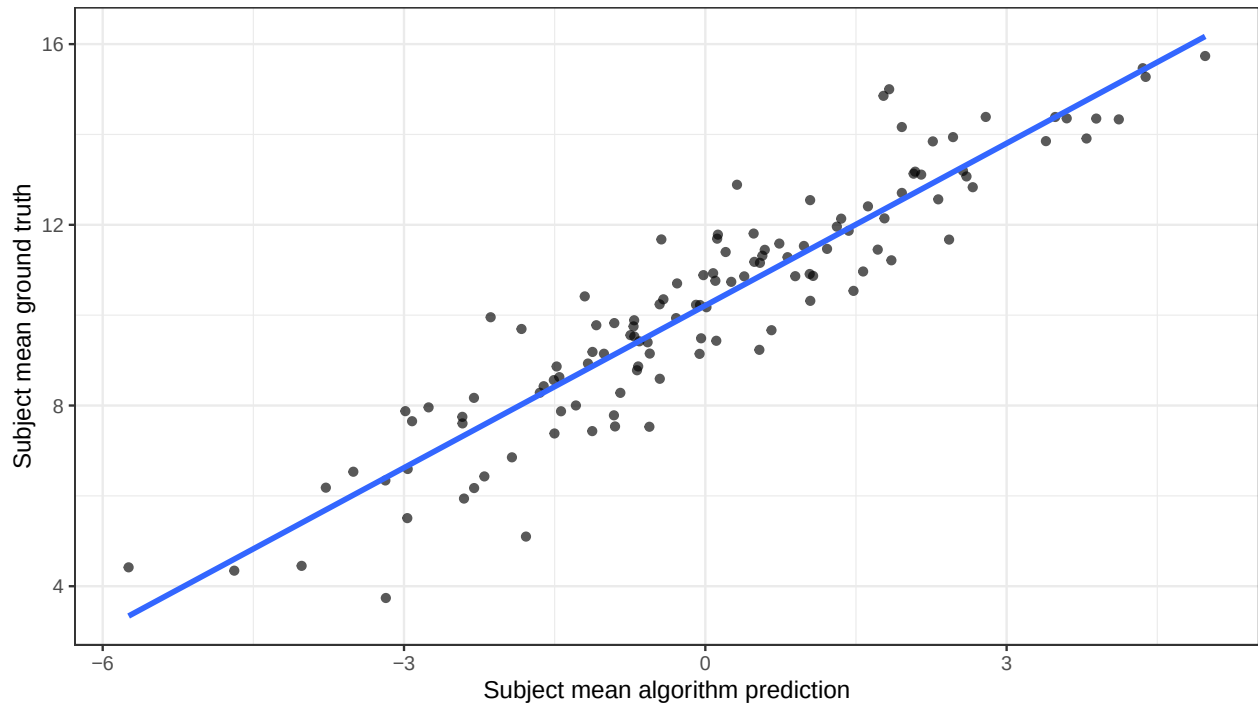
Each point is one subject's average X and average Y



```
plot_between_relationship(dat2, "Scenario 2: Between-subject association")
```

Scenario 2: Between-subject association

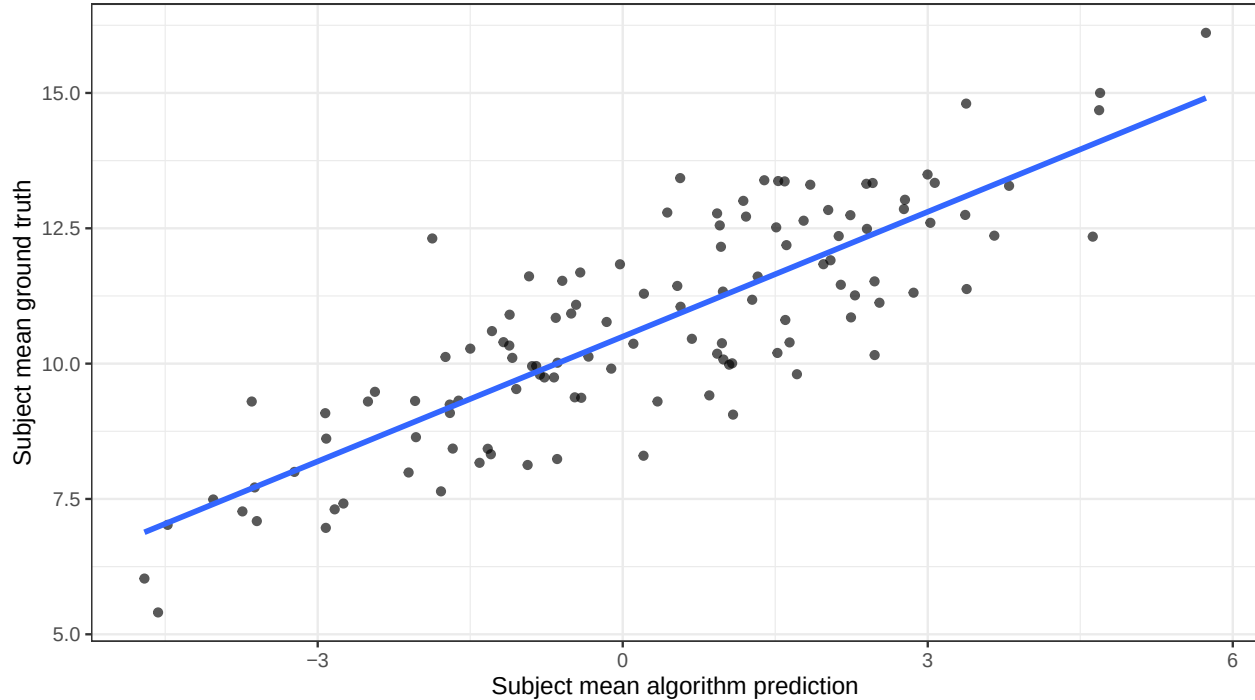
Each point is one subject's average X and average Y



```
plot_between_relationship(dat3, "Scenario 3: Between-subject association")
```

Scenario 3: Between-subject association

Each point is one subject's average X and average Y



Optional: compare unstandardized and standardized effects

This section illustrates why separate standardization changes the question.

```
calculate_standardized_effects <- function(dat, models, scenario_label) {  
  
  m_wb <- models$within_between  
  beta_B <- fixef(m_wb)["x_bar"]  
  beta_W <- fixef(m_wb)["x_within"]  
  
  sd_x_bar <- sd(unique(dat[, c("id", "x_bar")])$x_bar)  
  sd_x_within <- sd(dat$x_within)  
  
  data.frame(  
    scenario = scenario_label,  
    beta_B_raw = beta_B,  
    beta_W_raw = beta_W,  
    sd_x_bar = sd_x_bar,  
    sd_x_within = sd_x_within,  
    beta_B_per_1SD_x_bar = beta_B * sd_x_bar,  
    beta_W_per_1SD_x_within = beta_W * sd_x_within  
  )  
}  
  
standardized_effects <- bind_rows(  
  calculate_standardized_effects(dat1, models1, "1_opposite_direction"),  
  calculate_standardized_effects(dat2, models2, "2_same_direction_different_size"),  
  calculate_standardized_effects(dat3, models3, "3_same_direction_similar_size")  
)
```

```
print(standardized_effects)
```

```
##                                scenario beta_B_raw beta_W_raw sd_x_bar
## x_bar...1                1_opposite_direction  1.1654363 -0.7819864 2.040779
## x_bar...2 2_same_direction_different_size  1.1969749  0.2317959 2.050161
## x_bar...3  3_same_direction_similar_size  0.7684597  0.7750300 2.205479
##                sd_x_within beta_B_per_1SD_x_bar beta_W_per_1SD_x_within
## x_bar...1    0.9143389          2.378398          -0.7150006
## x_bar...2    0.8941028          2.453992           0.2072494
## x_bar...3    0.8952529          1.694821           0.6938478
```

The raw slopes answer:

Is one raw unit of X associated with the same change in Y between subjects and within subjects?

The standardized effects answer:

Is one standard deviation of between-subject X associated with the same change in Y as one standard deviation of within-subject X ?

These are related but different questions.

Scenario-specific interpretation

Scenario 1: Opposite direction

True data-generating values:

$$\beta_B = 1.2, \quad \beta_W = -0.8$$

This is the most dangerous case. The algorithm may separate high-burden from low-burden subjects, but it may fail to track within-person change correctly, or even track it in the wrong direction.

In this scenario, the pooled model is misleading because it compresses two opposing associations into one coefficient.

Scenario 2: Same direction, different size

True data-generating values:

$$\beta_B = 1.2, \quad \beta_W = 0.3$$

Both associations are positive, but the between-subject association is much stronger than the within-subject association.

This means the algorithm may rank subjects well, but it may be less sensitive to within-person change.

The pooled model may be directionally correct, but it hides an important difference between cross-sectional validity and longitudinal tracking validity.

Scenario 3: Same direction, similar size

True data-generating values:

$$\beta_B = 0.8, \quad \beta_W = 0.75$$

Here the between-subject and within-subject slopes are similar. If the estimated difference is small and the confidence interval excludes practically meaningful differences, the pooled model may be defensible.

Practical recommendation

For repeated-measure algorithm validation, a useful workflow is:

1. Fit the within-between model:

$$Y_{ij} = \beta_0 + \beta_B \bar{X}_i + \beta_W (X_{ij} - \bar{X}_i) + \gamma t_{ij} + b_i + \epsilon_{ij}$$

2. Report $\hat{\beta}_B$, $\hat{\beta}_W$, and $\hat{\beta}_B - \hat{\beta}_W$.
3. Use either the reparameterized model or `glht()` to test:

$$H_0 : \beta_B - \beta_W = 0$$

4. Fit the pooled model only if the between and within slopes are sufficiently similar.
5. Do not rely only on a non-significant p-value. Consider the confidence interval and whether the possible difference is practically meaningful for the intended algorithm use.